

Relatório de Desenvolvimento e Otimização do Solucionador Klotski

Para: Professor João Araújo **Disciplina:** Laboratório de Programação **Instituição:** Universidade do Estado do Rio de Janeiro (UERJ) **Data:** 28 de Novembro de 2025
Autor: Manus AI

1. Introdução

Este relatório detalha o processo de desenvolvimento e otimização de um solucionador para o Puzzle Klotski, conforme solicitado no documento **Haikori2025_2.pdf**. O objetivo principal foi criar um programa em linguagem C capaz de encontrar a sequência mínima de movimentos para resolver configurações arbitrárias do puzzle, com foco na eficiência algorítmica.

A metodologia de desenvolvimento seguiu uma abordagem iterativa, começando com uma solução base (Busca em Largura - BFS) e aplicando otimizações sucessivas para lidar com o vasto espaço de estados do problema. O uso de Inteligência Artificial (IA) foi uma ferramenta fundamental em todas as etapas do processo, desde a concepção inicial até a implementação das otimizações avançadas.

2. Solução Inicial: Busca em Largura (BFS) com Hash Set

A solução inicial foi concebida utilizando o algoritmo de **Busca em Largura (BFS)**, que garante a descoberta do caminho mais curto (solução ótima) em termos de número de movimentos.

2.1. Estrutura de Dados

Para evitar a re-exploração de estados e garantir a viabilidade da busca, foi implementado um **Hash Set** (tabela *hash* com encadeamento separado) para

armazenar os estados já visitados.

Componente	Função	Justificativa Técnica
State	Representação do tabuleiro e das peças.	Armazena a configuração atual do puzzle.
Node	Estrutura de BFS/A*.	Armazena o estado, o nó pai e o movimento que levou a este estado.
Queue	Fila FIFO.	Estrutura central para a exploração de estados em BFS.
HashTable	Hash Set de estados visitados.	Essencial para a poda de estados repetidos, transformando a busca de árvore em busca de grafo.

3. Otimizações de Desempenho

Três otimizações principais foram propostas e implementadas para aumentar a eficiência do solucionador, transformando a solução inicial de BFS em uma abordagem mais sofisticada.

3.1. Otimização 1: Representação do Estado e Hashing

A primeira otimização visou reduzir a sobrecarga de memória e o tempo de cálculo do *hash* de cada estado.

Alteração	Detalhe	Impacto
Representação Canônica	O estado passou a ser representado por um <i>array</i> de coordenadas (<i>row</i> , <i>col</i>) de cada peça, ordenado pelo ID da peça.	Reduz a memória necessária por estado e garante que o <i>hash</i> seja consistente.
Função de Hashing Otimizada	O <i>hash</i> é calculado diretamente sobre o <i>array</i> de coordenadas, eliminando a necessidade de gerar e <i>hashear</i> a <i>string</i> completa do tabuleiro.	Acelera significativamente a inserção e a consulta na tabela <i>hash</i> .

3.2. Otimização 2: Busca Bidirecional (Bi-BFS)

A Busca Bidirecional (Bi-BFS) foi implementada para explorar o espaço de estados simultaneamente a partir do estado inicial e do estado final.

Conceito	Detalhe	Vantagem
Bi-BFS	Duas buscas BFS independentes se expandem até se encontrarem em um estado comum.	Reduz drasticamente o número de nós a serem explorados, pois $O(b^{d/2} + b^{d/2})$ é muito menor que $O(b^d)$, onde b é o fator de ramificação e d é a profundidade da solução.
Estado Final	Implementação da função <code>create_goal_state</code> para gerar o estado final (peça 'X' na posição de saída) como ponto de partida para a busca reversa.	Permite que a busca reversa seja iniciada de forma precisa.
Colisão e Reconstrução	Lógica para detectar a colisão entre as duas buscas e reconstruir o caminho completo (caminho <i>forward</i> + caminho <i>backward</i> invertido).	Garante a solução ótima com a eficiência da Bi-BFS.

3.3. Otimização 3: Algoritmo A* com Heurística

A otimização final substituiu o BFS pelo algoritmo de **Busca A***, que utiliza uma heurística para guiar a busca de forma mais inteligente.

Alteração	Detalhe	Impacto
Estrutura Node	Adição dos campos g (custo real), h (custo heurístico) e f (custo total $g + h$).	Permite que o algoritmo priorize estados que estão mais próximos do objetivo.
Heurística (h)	Implementação da Distância de Manhattan da peça alvo ('X') até sua posição final.	Heurística admissível, garantindo que o A* encontre a solução ótima.
Priority Queue	Substituição da fila FIFO por uma Min-Heap (Fila de Prioridade) para sempre expandir o nó com o menor valor de f .	Acelera a convergência para a solução, pois a busca é guiada.
Poda de Movimentos	A poda de estados repetidos (já implementada com o <i>hash set</i>) é mantida, sendo a forma mais segura de poda para o A* no Klotski.	Evita ciclos e re-exploração de estados, mantendo a admissibilidade da heurística.

4. O Papel da Inteligência Artificial (IA) no Desenvolvimento

O desenvolvimento da solução foi um processo de **Programação Assistida por IA**, onde a Inteligência Artificial (o agente Manus) atuou como um engenheiro de software sênior, realizando as seguintes tarefas:

Etapa	Tarefa Executada pela IA	Metodologia de IA
Análise e Planejamento	Interpretação do PDF (Haikori2025_2.pdf) e dos arquivos de entrada, e criação do plano de desenvolvimento em fases.	Processamento de Linguagem Natural (PLN) e Raciocínio Lógico/Planejamento.
Concepção da Solução Base	Desenvolvimento do código-fonte inicial em C para o solucionador BFS com Hash Set.	Geração de Código e Conhecimento Específico de Domínio (Algoritmos de Busca e Estruturas de Dados).
Proposta de Otimizações	Análise do código base e sugestão das 3 otimizações (Representação de Estado, Bi-BFS, A*).	Análise de Código e Raciocínio Algorítmico Avançado.
Implementação Iterativa	Implementação das otimizações, incluindo a criação de novas estruturas de dados (Priority Queue) e funções (Heurística, Bi-BFS).	Geração de Código Iterativa e Refatoração de Código.
Depuração e Correção	Identificação e correção de erros de compilação (ex: erro de <i>linker</i> <code>print_state</code> , erro de atribuição de <i>array</i> em C).	Análise de Erros e Depuração de Código.
Documentação	Elaboração deste relatório e do <i>Code Review Report</i> anterior.	Geração de Texto Técnico e Síntese de Informação.

O processo demonstrou a capacidade da IA de não apenas gerar código, mas de **analisar, otimizar e refatorar** soluções complexas de forma iterativa, seguindo uma metodologia de engenharia de software rigorosa.

5. Conclusão

O solucionador Klotski evoluiu de uma implementação BFS básica para uma solução A* guiada por heurística, incorporando otimizações críticas de representação de estado e estruturas de dados. A intervenção da Inteligência Artificial foi essencial para acelerar o ciclo de desenvolvimento e garantir a aplicação de técnicas algorítmicas

avançadas, resultando em um código robusto e eficiente para a resolução do problema.

Anexos:

- `klotski_solver_a_star.c` (Código-fonte final)
- `klotski_solver_bidirectional.c` (Código-fonte da Otimização 2)
- `klotski_solver_optimized.c` (Código-fonte da Otimização 1)
- `klotski_solver.c` (Código-fonte inicial)
- `Code_Review_Report.pdf` (Relatório de Revisão de Código anterior)